FULL STACK DEVELOPER PATH

# Chapter 5 Databases & SQL

*Structured · Queryable · data that outlives the app*

columns = fields, each with a fixed type

| id | brand | model | year | price | sold |
|---|---|---|---|---|---|
| 1 | Ford | Mustang | 1965 | 15000 | false |
| 2 | Porsche | 356B | 1962 | 280000 | true |
| 3 | Bentley | S2 | 1959 | 62000 | false |

rows = records

**SOURCE :** freeCodeCamp × Scrimba — "Become a Fullstack Developer from Scratch"

**MODULES :** Introduction to databases (24:52:41) · Writing SQL queries (25:13:25)

**COVERS :** SQL vs NoSQL · ORMs · hosting · SELECT · WHERE · ORDER BY · aggregates

**PROJECT :** Retro Rides — a vintage car dealership, powered by PGlite

⚡ THE KEY IDEA

A relational database stores data in tables of rows and columns with a **rigid, predefined schema**. SQL is the language you use to ask it questions — and the core syntax is shared by every flavour.

## 1 Beyond the table ★★★

Relational tables are not the only way to store data. The course surveys several **NoSQL** shapes, each suited to a different job:

| Type | How data is stored | Good for |
|---|---|---|
| **Document** | Documents whose fields may differ from one another | Flexible, evolving records |
| **Key-value** | A key mapped directly to a value | Fast, simple lookups |
| **Column family** | Data in columns, columns grouped into **column families** | Time-series data, logging, event tracking |
| **Graph** e.g. Neo4j | **Nodes** (entities) and **edges** (the relationships between them) | Complex relationships — e.g. a social network, where the key functionality is the relationships |

DEFINITION — SCHEMA

The schema of an SQL database is *rigid and predefined*. You define it when you define the database, and that ensures any data you try to add **must fit the pattern you predefined**. NoSQL databases can be **schemaless**, or have dynamic schemas based on the data included.

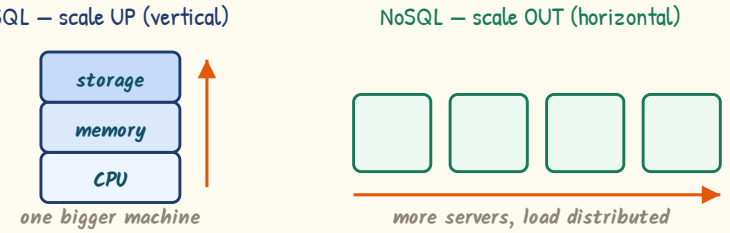



Fig 1 — the two directions a database can grow

QUERY LANGUAGES

There are different **flavours** of SQL, but all of them rely on the *core SQL syntax*. NoSQL query languages vary a lot — a MongoDB query, Cypher, CQL — depending on which system you're using. MongoDB queries look **very similar to writing JavaScript**.

⚡ PICK THE SHAPE, NOT THE FASHION

Each NoSQL type exists because a particular *access pattern* is awkward in tables. Column families suit append-heavy logs; a graph database suits a social network because there the **relationships are the product**. Choosing one because it is newer than SQL is not a reason.

## 2 Choosing between them ★★★★

| | SQL | NoSQL |
|---|---|---|
| Data structure | Fixed and rigid. Each column has a specific data type and may carry constraints | More flexible — documents can have different field values |
| Storage | Tables of rows and columns | Documents, key-value pairs, graphs, columns |
| Schema | Rigid and predefined | Schemaless, or a dynamic schema |
| Scalability | Vertical — scale up by adding CPU, memory, storage | Horizontal — scale out by adding servers or instances |
| Query language | Different flavours, but all share the core SQL syntax | Varies by system — MongoDB queries, Cypher, CQL |
| Use cases | Structured data; complex relationships between tables | Large-scale; unstructured or semi-structured data |
| Prioritises | Consistency of data | Flexibility and scale |

⚡ THE ONE-LINE VERDICT

Reach for SQL when you want to prioritise the consistency of data and you have complex relationships between pieces of data. Reach for NoSQL when you want greater flexibility and scale — which suits modern web applications that don't require strict structure and relationships.

⚠ NOT A RANKING

Neither column is the "advanced" one. The rigidity of SQL is a feature — it is precisely what guarantees that bad data cannot get in. The flexibility of NoSQL is the same trade made in the opposite direction.

### ▶ Constraints — the other half of "rigid"

Saying each column has a specific data type understates it: a column may also carry constraints. Together these are what the database enforces on your behalf, before any application code runs.

- A `year` column typed as a number cannot receive the string `"unknown"`.
- A `sold` boolean has exactly two legal values.
- A primary key is guaranteed unique.
- Bad data is rejected at the door, not discovered later.

⚡ WHAT "CONSISTENCY" ACTUALLY BUYS YOU

Every row in a table is guaranteed to have the same shape. That is why a query can safely assume `price` is a number on all 41 rows — and why aggregates like `SUM()` and `AVG()` are meaningful at all. In a schemaless store you would have to defend against missing or mistyped fields in your own code.

## 3 Application structure ★★★★


*the API sends information back and forth to the database*

Fig 2 — the three parts of a typical application

DEFINITION — RDBMS

Your point of interaction with the database is usually a **relational database management system** — such as **Postgres** — used to create and interact with the database. Various libraries implement databases inside applications: **SQLite**, **SQLAlchemy**, and the **PGlite** library used in this course.

## 4 Object-relational mapping ★★★

An **ORM** lets you take an object-oriented approach rather than directly writing SQL queries. You write in JavaScript or Python; the ORM translates that into database queries.


```
// Prisma — an ORM. Queries look like calling a method in JavaScript
const user = await prisma.user.findFirst({
    select: { email: true, name: true }
});
```

| Use an ORM when… | Write SQL yourself when… |
|---|---|
| Developers don't already know SQL, or want to avoid learning complex SQL | You're building a **complex** application |
| The team runs a fairly simple application and doesn't need bespoke queries | You need full control over the database and much more customisability |

⚡ THE TRADE AN ORM MAKES

An ORM buys you convenience and a familiar language and charges you in **control**. You get data back as objects ready to populate a front end, without writing bespoke SQL — but the query that actually reaches the database is one **you did not write**. Learning SQL first is what lets you judge whether that generated query is any good.

# 5 Managed vs self-hosted ★★★

DEFINITION — DATABASE ADMINISTRATION

A job role and a broad set of responsibilities covering the management of a **production** database. The person in this role will create or drop tables, manage access to them, and otherwise control the data — a complex role, with the security of the database and the integrity of its data both in scope.

| | Managed | Self-hosted |
|---|---|---|
| What it is | A cloud-based solution provided by a **third party** — you outsource the hosting | Run on infrastructure owned **in house** — a server, VM or Docker container |
| Benefits | Flexibility to scale as requirements grow · costs optimised by paying gradually more · removes the burden of maintaining security and integrity yourself | Customise the setup any way that suits you · add your own custom layers of security · no reliance on third-party management |
| Simple setup | Quick start, often more cost-effective | Overheads you may not need |
| At scale | Can become quite costly; heavy reliance on external management | Generally more cost-effective |

WHEN SELF-HOSTING WINS EARLY

A complex setup from day one — where you need full customisation and high levels of security. The worked example is **financial data**: you want to be absolutely sure the database is watertight and would rather not rely on a third party, because losing data is terrible for your users and in turn for you.

⚡ HOW TO ANSWER THIS ONE

The trade is convenience now vs cost and control later. Small database, simple structure → managed. Large, sensitive or highly customised → self-hosted. There is no universally correct answer, and saying so **is** the correct answer.

## ▶ What "self-hosted" actually means in practice

- You provision the **server** — or a virtual machine, or a **Docker container**.
- You own the security layers on top of it.
- You need someone doing **database administration**.
- Nothing about the setup is a mystery to you.

⚠ THE HIDDEN COST OF MANAGED

Outsourcing removes the burden of maintaining security and integrity — but it also means relying on a third party for exactly that. In a regulated or sensitive industry that reliance is itself the risk you are trying to eliminate, which is why the financial-data example lands on **self-hosted** despite the extra work.

## 6 The project & the table ★★★★

*Retro Rides* is a car dealership selling vintage cars from the 20th century. The `cars` table holds *41 records*:

| Column | Type | Notes |
|---|---|---|
| `id` | number | Unique, incremented *sequentially* |
| `brand`, `model` | text | – |
| `year`, `price` | number | – |
| `color` | text | Shades vary – "light yellow" is not "yellow" |
| `condition` | number | Scale 0–5. 5 = pristine, 0 = a total wreck |
| `sold` | boolean | `true` or `false` |

THE SETUP

A simple Node app running from one `index.js`, using *PGlite* – an implementation of PostgreSQL, which is very common across the industry and integrates nicely with JavaScript. Queries live in a `.sql` file; saving runs it and logs the output. Setting up tables and building a database could be a whole course in itself, so this module focuses on the commands.

### ▶ Your first query

```
-- every column
SELECT *
FROM cars;

-- just some columns
SELECT brand, model, price
FROM cars;
```

- SQL centres on *keywords* – here `SELECT` and `FROM`.
- `*` is the *wildcard* operator – every column.
- Separate column names with commas.
- End every statement with a semicolon – that tells SQL the statement is ready to run.

⚡ CAPITALS ARE A CONVENTION, NOT A RULE

SQL generally uses *CAPITALS for keywords* like `SELECT` and `FROM`, and lowercase for column and table names. This isn't strictly necessary – all caps, all lowercase or the convention will each run fine. Following it just makes statements more readable. SQL also *doesn't worry about spacing*, so indent freely.

WHY FILTER THE COLUMNS AT ALL

`SELECT *` is fine while exploring, but naming the columns filters the output to only what is relevant – you stop seeing the colour and condition when all you wanted was brand, model and price. On a real table that is also *less data over the wire*.

## 7 Conditions ★★★★★

```
SELECT brand, model, color, price
FROM cars
WHERE color = 'black';
```

| Operator | Meaning |
|---|---|
| `=` | Exact match — use for strings and numbers |
| `>` `<` | Greater than · less than |
| `>=` `<=` | ...or equal to |
| `!=` | Not equal to |
| `<>` | Also not equal to — the same result, you may meet it in other people's SQL |
| `IS` | Use for boolean or NULL values |

⚠ = vs IS — THE EDGE CASE

Usually these give the same result and it wouldn't matter which you use. But there are edge cases with boolean values, and writing `IS FALSE` avoids them. So: = for strings and numbers, IS for booleans and NULL.

### ▶ Partial matches — LIKE

```
-- any shade of red
WHERE color LIKE '%red%'
```

`=` only finds exact matches — filtering out `'yellow'` leaves "light yellow" in your results. `LIKE` with `%` matches anything before or after.

### ▶ Sets — IN

```
WHERE brand IN ('Ford', 'Chevrolet', 'Ferrari')
  AND sold IS FALSE;

-- and its negation
WHERE brand NOT IN ('Ford', 'Chevrolet', 'Dodge')
```

WHY IN IS WORTH LEARNING

`IN` lets one column match any of several values — without writing multiple `OR` clauses. `BETWEEN 1960 AND 1969` does the same job for a numeric range.

⚡ HOW MUCH FEEDBACK SHOULD A FILTER GIVE?

If a customer asks for a continent — or a colour — that isn't in the table, they simply get nothing back. That is a design decision, not an accident: how much feedback you give someone who misuses a query or an endpoint is something to think about deliberately, exactly as it was when designing the Node API in Chapter 4.

## 8 AND, OR and the bracket trap ★★★★★

A customer will spend under $250,000 — or more, if it's a Porsche. That is an **OR**. Now add: "but only cars in good condition". This is where it goes wrong.

```sql
-- ✗ WRONG
WHERE price < 250000
   OR brand = 'Porsche'
  AND condition > 3;
```

```sql
-- ✓ RIGHT
WHERE (price < 250000
   OR brand = 'Porsche')
  AND condition > 3;
```

⚠ THE SILENT WRONG ANSWER

Run the first version and nothing appears to change — no error, just wrong rows. Cars come back with a condition of 1 or 2 still in the list.

SQL reads it as: price under 250,000 **OR** (brand is Porsche **AND** condition > 3). The **AND** binds tighter, so the condition check only ever applies to the Porsches. Brackets force your meaning.

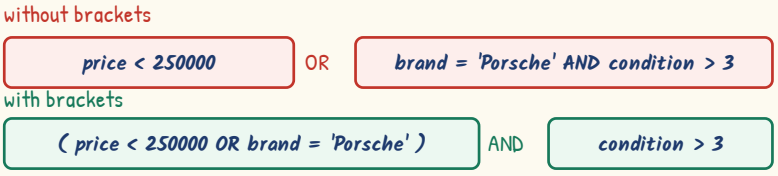



Fig 3 — the same three conditions, grouped two different ways

```sql
-- a Dodge from the '60s, OR a Ford/Triumph from the '70s — and never a sold car
WHERE (
        (brand = 'Dodge' AND year BETWEEN 1960 AND 1969)
     OR (brand IN ('Ford', 'Triumph') AND year BETWEEN 1970 AND 1979)
      )
  AND sold IS NOT TRUE;
```

⚡ BUILD IT ONE STEP AT A TIME

Complex conditions are assembled, not written in one go: get the first clause working, add the **OR**, then wrap the pair in brackets and hang the universal condition off the end. **IS NOT TRUE** and **IS FALSE** give the same result set here.

THE BUG YOU ONLY CATCH BY READING RESULTS

Nothing in the tooling will tell you the brackets were missing. The only symptom is a row that should not be there — an Alfa Romeo Spider or a Ford Mustang with a condition of 1 sitting in a list that was supposed to be filtered to 4s and 5s. Get into the habit of spot-checking a few returned rows against every clause you wrote.

## 9 ORDER BY ★★★★

```
SELECT brand, model, condition, price
FROM cars
WHERE sold IS FALSE
ORDER BY condition DESC, price
LIMIT 5;
```

| Point | Detail |
|---|---|
| Default direction | Ascending — alphabetical for strings, lowest-to-highest for numbers |
| `DESC` | Short for descending, reverses it |
| `ASC` | Exists, but is already the default — you don't need to write it |
| Several columns | Separate with commas — sorts by the first, then breaks ties with the second |
| Clause order | `WHERE` goes after FROM, before ORDER BY |
| `LIMIT n` | Caps the number of rows returned. Goes at the end |

ORDER BY + LIMIT = "THE MOST …"

There is no "max row" keyword. To find the single most expensive car you sort by price descending and limit to 1. Sort ascending and limit to 5 for the five cheapest. This pairing is one of the most useful idioms in SQL.

## 10 Aggregates ★★★★★

DEFINITION

Aggregations turn the values within a single column from multiple values into one single value — counting records, or totalling a set of values.

```
SELECT COUNT(*) AS total_sold
FROM cars
WHERE sold IS TRUE;

SELECT SUM(price) AS total_earnings
FROM cars WHERE sold IS TRUE;
```

| Function | Returns |
|---|---|
| `COUNT()` | Number of rows |
| `SUM()` | Total of a column |
| `MAX()` `MIN()` | Largest · smallest |
| `AVG()` | The average |
| `FLOOR()` `CEIL()` | Round down · up |

⚠ YOU CANNOT SELECT OTHER COLUMNS ALONGSIDE

Although you get one value back, you may not put an ordinary column next to it. What looks like selecting a single value is really selecting a column and reducing it to one value. To see other columns beside an aggregate you must `GROUP BY` — next page. Use `AS` to alias the output column a readable name, and wrap an aggregate in `FLOOR()` or `CEIL()` to lose the trailing decimals.

## 11 Grouping results ★★★★★

**GROUP BY** collects rows into **matching sets**, then runs the aggregate **once per set** rather than once for the whole table.

```
SELECT brand, COUNT(brand) AS count, FLOOR(AVG(price)) AS avg
FROM cars
WHERE sold IS FALSE
GROUP BY brand;
```

READ IT AS

"Rather than counting the total number of cars in the table, count **for each specific brand**." Because the rows are grouped, **brand** may now appear beside the aggregate — that is exactly the restriction **GROUP BY** lifts.

### ▶ HAVING — a WHERE for aggregates

⚠ WHY HAVING EXISTS

Using an **aggregate column in a WHERE clause will throw an error.** **HAVING** was added to SQL specifically to allow conditions that use aggregations.

```
SELECT year, COUNT(year) AS car_count, MAX(price), MIN(price)
FROM cars
WHERE sold IS TRUE          -- filters individual rows
GROUP BY year
HAVING COUNT(year) > 1      -- filters whole groups
ORDER BY car_count;
```

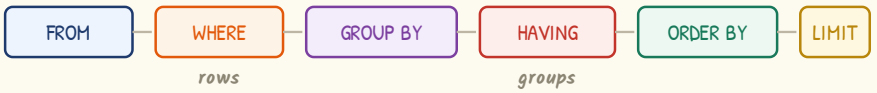



Fig 4 — clause order, and what each filter acts on

⚡ WHERE FILTERS ROWS · HAVING FILTERS GROUPS

That single sentence is the whole distinction. **WHERE** runs **before** grouping and decides which rows enter a group; **HAVING** runs **after** and decides which finished groups survive.

WHY YOU USUALLY WANT HAVING

Group 41 cars by brand and the one-off brands — a single Jaguar, a single Lamborghini — each produce a group of one. Their "average price" is just that car's price, which **isn't really telling us much**.

**HAVING COUNT(brand) > 1** drops them, and every remaining row **says something.**

## 12 CRUD ★★★★

DEFINITION — DML

SQL's **data manipulation language** commands are also known as **CRUD** commands: Create, Read, Update, Delete — the four main operations for manipulating data. If you have worked with APIs you will have met these terms before. You have already been using one of them: **read** is what a `SELECT` statement does.

| Letter | SQL keyword | Note |
|---|---|---|
| Create | `INSERT INTO` | Add new records |
| Read | `SELECT` | Already covered — pages 6–10 |
| Update | `UPDATE ... SET` | Modify existing records |
| Delete | `DELETE FROM` | Remove records |

THEY RETURN NOTHING

Unlike a `SELECT`, these commands don't give you a result set back. In the course project the setup is changed so it executes the command and **then runs a separate SELECT**, purely so you can visualise what the insert, update or delete actually did.

### ▶ INSERT INTO

```
INSERT INTO cars (brand, model, year, price, color, condition, sold)
VALUES
    ('Ford', 'Escort RS2000', 1978, 39000, 'blue', 4, false),
    ('Aston Martin', 'V8', 1977, 185000, 'silver', 4, false);
```

⚠ SKIP THE ID

The `id` is the **primary key** and is auto-incremented — adding a record automatically assigns the next value in the sequence. It is the one column you do **not** supply.

⚠ FILL EVERY OTHER COLUMN

Insert data into every remaining column. That avoids adding NULL values, which can cause database errors later.

Name the columns in brackets after the table, then list the matching data after the `VALUES` keyword — **in the same order**. Separate multiple rows with commas.

DEFINITION — PRIMARY KEY

The column that uniquely identifies a row. In the `cars` table that is `id`, described as unique and incremented sequentially. Because the database assigns it, two records can never collide — and you never have to work out **what the next number should be**.

## 13 ⚡ QUICK REVISION

| Thing | The one-line answer |
|---|---|
| SQL schema | **Rigid and predefined** — data must fit the pattern you set up front |
| SQL scaling | **Vertical** (bigger machine). NoSQL scales **horizontally** (more machines) |
| Pick SQL when… | Consistency matters and relationships are complex |
| RDBMS | Relational database management system, e.g. **Postgres** |
| ORM | Write JavaScript, it emits SQL. Prisma is the example. Less control |
| Statement ends | With a **semicolon**. Spacing and indentation are free |
| Capitals | Convention only — nothing breaks if you ignore it |
| `*` | The wildcard — every column |
| `=` vs `IS` | `=` for strings/numbers · `IS` for booleans and NULL |
| Not equal | `!=` or `<>` — identical meaning |
| Partial match | `LIKE '%red%'` — `=` would miss "light yellow" |
| Many values | `IN (...)` beats stacking `OR`s · `BETWEEN a AND b` for ranges |
| **Brackets** | `AND` binds tighter than `OR` — group them or get wrong rows **silently** |
| Default sort | Ascending. `DESC` reverses; `ASC` is redundant |
| "The most expensive" | `ORDER BY price DESC LIMIT 1` |
| Aggregate alone | Can't sit beside a normal column — unless you `GROUP BY` |
| `AS` | Aliases the output column to a readable name |
| WHERE vs HAVING | **WHERE filters rows, HAVING filters groups.** Aggregates in WHERE → error |
| CRUD | Create · Read · Update · Delete = `INSERT` · `SELECT` · `UPDATE` · `DELETE` |
| Inserting | Omit the auto-incremented **primary key**; fill every other column to avoid NULLs |

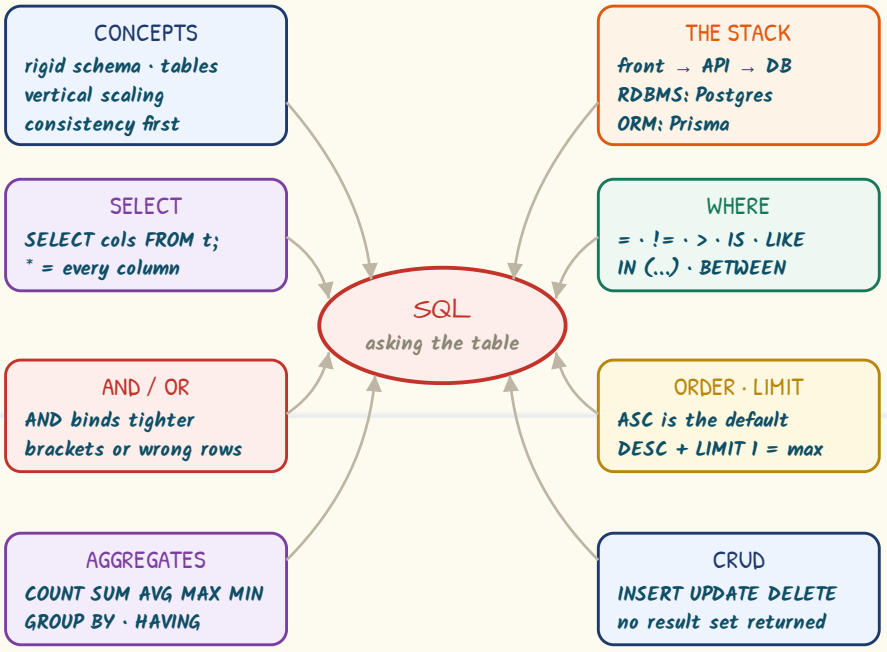
### ⚡ THE FIVE MOST FORGETTABLE DETAILS

1. Missing brackets around an `OR` return wrong rows with no error at all. 2. `=` on a boolean hits edge cases — use `IS`. 3. `LIKE` is needed whenever the data has shades and variants. 4. An aggregate in `WHERE` throws; it belongs in `HAVING`. 5. Never insert the `id` yourself.

### THE SKELETON — MEMORISE THE ORDER

```
SELECT columns, AGGREGATE(col) AS alias
FROM   table
WHERE  row-level conditions
GROUP BY column
HAVING  group-level conditions
ORDER BY column DESC
LIMIT  n;
```

## 14 MIND MAP




⚡ THE THROUGH-LINE

Every query is the same skeleton with optional pieces: SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT. Learn that spine and the rest is vocabulary. The one thing that will bite you silently is **operator precedence** — when in doubt, add brackets.

| Term | One line |
|---|---|
| RDBMS | The system you talk to — Postgres in this course |
| PGlite | The library used in the scrims; an implementation of PostgreSQL |
| Schema | The predefined shape every row must fit |
| Primary key | Unique per row, auto-incremented — never inserted by hand |
| Alias | A readable output name, set with `AS` |
| ORM | Writes the SQL for you, from JavaScript objects |

Next up → **React**, where this data finally reaches a user interface.